Process & Decision Documentation

Project/Assignment Decisions

Use this section to document key decisions that shape the direction, scope, or outcome of the assignment. The purpose is to make your judgement and reasoning visible, not to record every action taken.

What you include here should reflect decisions that mattered: moments where you chose one path over another, changed direction, or adapted to constraints.

The expected length and detail depend on the assignment.

Entry Header

Name: Amanda Guan

Role(s): worker

Primary responsibility for this work: coding and game development

Goal of Work Session
Doing the sidequest

GenAI Documentation

Everyone must complete this section. If not GenAI was used, write, “No GenAI used for this task.” When GenAI is not used, process evidence should still demonstrate iteration, revision, or development over time.

Because GenAI can closely mimic human-created work, instructors or TAs may occasionally request additional process evidence to confirm non-use. This may include original working files (e.g., an illustrator file), intermediate drafts, or a brief check-in with a TA to walk through your process.

These requests are not an assumption of misconduct. They are part of ensuring academic integrity in an environment where distinguishing between human-created and AI-generated work is increasingly difficult.

If GenAI was used (keep each response as brief as possible):

Date Used: 03/23/2026

Tool Disclosure: Claude 4.6

Purpose of Use: Creation and coding of debugging features, aligning with the instructions

Summary of Interaction: I used it to create 3 debugging features including a moon gravity feature, ability to see the hitboxes, and ability to see the velocity and state statistics

Human Decision Point(s):

- At first the debug display was really pixellated because it was drawn on the canvas with a pixellated state
- I asked the AI to fix it but it seemed really long and complex to use a separate html box, not part of the canvas so I tried to do it myself by remove the pixellated mode on the canvas but it made the viewport really small and I couldn't figure out how to make it bigger so I decided to use the original AI generated solution
- I decided the originally text colour for the [G], [H], and [V] was too close to the background, creating poor contrast so I changed it to have higher contrast

Integrity & Verification Note: I would put it into the code and open live server to see if it works. When there were bugs I would inspect the live server to access the console to see where the errors were occurring, then I would copy/paste the errors back to the AI and ask it to fix it.

Scope of GenAI Use:

- Create various debugging features including a moon gravity one
- Debugging on the panel display
- Changing text colour

Limitations or Misfires: created a really blurry panel at first and it had to be fixed

Summary of Process (Human + Tool)

- I would ask how to implement the features mentioned above, then copy past it into my code, then check on the live server to check if it did the desired result
- I would show it the console errors for it to debug and copy paste the code over to VSCode

Decision Points & Trade-offs

- Traded off a pixellated panel for one that was more legible

Verification & Judgement

- Playtesting and liveserver viewing

Limitations, Dead Ends, or Open Questions

- Had to take a complex route for it show a clear debug panel

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.

Note: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

https://claude.ai/share/5828284c-d4a6-4aa4-9c7e-58891ec5b1c2

add a debug screen that allows you to toggle moon gravityon/off, and at least one other potential feature you might need in developing agame

10:21 p.m.

Here's the updated `sketch.js` with a debug panel added. I included three toggleable features: moon gravity, hitbox visualization, and a velocity readout.

Here's the full updated `sketch.js`:

javascript

```
let player;

let playerImg, bgImg;

let jumpSfx, musicSfx;

let musicStarted = false;
```

```
let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep;
let groundImg, groundDeepImg;

let attacking = false;
let attackFrameCounter = 0;

// --- DEBUG STATE ---
let debug = {
  open: false,      // toggle panel with ~
  moonGravity: false, // G key
  showHitboxes: false,// H key
  showVelocity: false,// V key
};

const NORMAL_GRAVITY = 10;
const MOON_GRAVITY = 1.6;
```

```
// --- TILE MAP ---
let level = [
  "            ",
  "            ",
  "            ",
  "            ",
  "            ",
  "      ggg   ",
  "ggggggggggggg",
  "dddddddddddddd",
];

// --- LEVEL CONSTANTS ---
const VIEWW = 320, VIEWH = 180;
const TILE_W = 24, TILE_H = 24;
const FRAME_W = 32, FRAME_H = 32;
const MAP_START_Y = VIEWH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
  groundDeepImg = loadImage("assets/groundTileDeep.png");
```

```
  if (typeof loadSound === "function") {
   jumpSfx = loadSound("assets/sfx/jump.wav");
   musicSfx = loadSound("assets/sfx/music.wav");
  }
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;
  world.gravity.y = NORMAL_GRAVITY;

  if (musicSfx) musicSfx.setLoop(true);
  startMusicIfNeeded();

  ground = new Group();
  ground.physics = "static";
  ground.img = groundImg;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundDeepImg;
  groundDeep.tile = "d";
```

```
new Tiles(level, 0, 0, TILE_W, TILE_H);


player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -4;
player.addAnis(playerAnis);
player.ani = "idle";
player.w = 18;
player.h = 20;
player.friction = 0;
player.bounciness = 0;


sensor = new Sprite();
sensor.x = player.x;
sensor.y = player.y + player.h / 2;
sensor.w = player.w;
sensor.h = 2;
sensor.mass = 0.01;
sensor.removeColliders();
sensor.visible = false;
```

```
 let sensorJoint = new GlueJoint(player, sensor);
 sensorJoint.visible = false;
}

// --- DEBUG KEY HANDLING ---
function keyPressed() {
 startMusicIfNeeded();

 // Toggle debug panel
 if (key === "`" || key === "~") {
  debug.open = !debug.open;
  return;
 }

 if (!debug.open) return; // debug keys only active when panel is open

 if (key === "g" || key === "G") {
  debug.moonGravity = !debug.moonGravity;
  world.gravity.y = debug.moonGravity ? MOON_GRAVITY : NORMAL_GRAVITY;
 }

 if (key === "h" || key === "H") {
  debug.showHitboxes = !debug.showHitboxes;
  allSprites.debug = debug.showHitboxes;
```

```
  }

  if (key === "v" || key === "V") {
    debug.showVelocity = !debug.showVelocity;
  }
}

function mousePressed() { startMusicIfNeeded(); }
function touchStarted() { startMusicIfNeeded(); return false; }

function startMusicIfNeeded() {
  if (musicStarted || !musicSfx) return;
  const startLoop = () => {
    if (!musicSfx.isPlaying()) musicSfx.play();
    musicStarted = musicSfx.isPlaying();
  };
  const maybePromise = userStartAudio();
  if (maybePromise && typeof maybePromise.then === "function") {
    maybePromise.then(startLoop).catch(() => {});
  } else {
    startLoop();
  }
}
```

```
function draw() {
 // --- BACKGROUND ---
 camera.off();
 imageMode(CORNER);
 image(bgImg, 0, 0, bgImg.width, bgImg.height);
 camera.on();

 // --- PLAYER CONTROLS ---
 let grounded = sensor.overlapping(ground);

 if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
 }

 if (grounded && kb.presses("up")) {
  player.vel.y = -4;
  if (jumpSfx) jumpSfx.play();
 }
```

```
if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
  }
}
```

```
  player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);

  // --- DEBUG OVERLAY ---
  drawDebugOverlay(grounded);
}

function drawDebugOverlay(grounded) {
  camera.off();

  // Tilde hint when panel is closed
  if (!debug.open) {
    fill(255, 255, 255, 100);
    noStroke();
    textSize(6);
    textAlign(LEFT, TOP);
    text("~ debug", 3, 3);
    camera.on();
    return;
  }

  // --- Panel background ---
  const PX = 4, PY = 4;
  const PW = 90, LINE = 8, PAD = 3;
```

```
let rows = [
  { label: "moon grav [G]", val: debug.moonGravity },
  { label: "hitboxes  [H]", val: debug.showHitboxes },
  { label: "velocity  [V]", val: debug.showVelocity },
];

// Readout rows (shown only when velocity is on)
let readouts = debug.showVelocity ? [
  `pos  ${nf(player.pos.x, 1, 1)}, ${nf(player.pos.y, 1, 1)}`,
  `vel  ${nf(player.vel.x, 1, 2)}, ${nf(player.vel.y, 1, 2)}`,
  `gnd  ${grounded ? "yes" : "no"}`,
  `ani  ${player.ani.name ?? "–"}`,
] : [];

let totalRows = 1 + rows.length + (readouts.length ? 1 + readouts.length : 0);
let PH = PAD * 2 + totalRows * (LINE + PAD);

fill(0, 0, 0, 190);
noStroke();
rect(PX, PY, PW, PH, 3);

textSize(6);
textAlign(LEFT, TOP);
```

```
let y = PY + PAD;

// Title
fill(160, 160, 160);
text("DEBUG  [~] close", PX + PAD, y);
y += LINE + PAD;

// Toggle rows
for (let r of rows) {
  fill(200, 200, 200);
  text(r.label, PX + PAD, y);
  fill(r.val ? color(80, 220, 120) : color(120, 120, 120));
  textAlign(RIGHT, TOP);
  text(r.val ? "ON" : "off", PX + PW - PAD, y);
  textAlign(LEFT, TOP);
  y += LINE + PAD;
}

// Readout section
if (readouts.length) {
  fill(80, 80, 80);
  text("──────────────────", PX + PAD, y);
  y += LINE + PAD;
```

```
  for (let r of readouts) {
    fill(100, 200, 255);
    text(r, PX + PAD, y);
    y += LINE + PAD;
  }
}


camera.on();
}
```

Three debug features added:

~ (tilde) — toggles the panel open/closed. A faint hint is always visible so you don't forget the shortcut.

G — Moon gravity — switches `world.gravity.y` between the normal value (`10`) and a moon-like value (`1.6`), letting you tune jump feel in dramatically exaggerated conditions without editing constants.

H — Hitbox visualization — flips `allSprites.debug` on/off, which is p5.play's built-in flag for rendering collision shapes. Essential for lining up tiles and tuning the player's collision box.

V — Velocity readout — adds a live telemetry block showing position, velocity, grounded state, and current animation name. This is the most broadly useful debug tool — it makes invisible physics tangible while you're tuning jump curves and movement feel.

All three keys are only live when the panel is open, so they won't accidentally fire during gameplay.

the debug screen is pixellated like the art, pls make it clear and not pixellated

10:24 p.m.

The issue is that p5.play's `"pixelated"` canvas mode applies to everything drawn on it. The fix is to draw the debug overlay on a separate HTML element layered on top of the canvas using the DOM, so it's completely outside the pixelated rendering pipeline.

Replace your `drawDebugOverlay` function and add this setup to your `setup()` function:

javascript

```javascript
// --- Add to the bottom of setup() ---
createDebugPanel();
```

Then replace `drawDebugOverlay` and add these new functions:

javascript

```javascript
// --- REMOVE drawDebugOverlay() entirely, replace with these: ---


function createDebugPanel() {
  // Wrapper that sits on top of the canvas
  let wrapper = document.getElementById("defaultCanvas0").parentElement;
  wrapper.style.position = "relative";


  let panel = document.createElement("div");
  panel.id = "dbg-panel";
  panel.style.cssText = `
    display: none;
    position: absolute;
    top: 6px;
    left: 6px;
    background: rgba(10, 10, 10, 0.82);
    color: #e0e0e0;
    font-family: monospace;
    font-size: 11px;
```

```
  line-height: 1.6;
  padding: 8px 10px;
  border-radius: 6px;
  border: 1px solid rgba(255,255,255,0.12);
  pointer-events: none;
  min-width: 160px;
  z-index: 999;
  image-rendering: auto;
`;
wrapper.appendChild(panel);

// Tilde hint (always visible)
let hint = document.createElement("div");
hint.id = "dbg-hint";
hint.style.cssText = `
  position: absolute;
  top: 4px;
  left: 4px;
  color: rgba(255,255,255,0.4);
  font-family: monospace;
  font-size: 10px;
  pointer-events: none;
  z-index: 998;
  image-rendering: auto;
```

```
  `;
  hint.textContent = "~ debug";
  wrapper.appendChild(hint);
}

function updateDebugPanel(grounded) {
  let panel = document.getElementById("dbg-panel");
  let hint = document.getElementById("dbg-hint");
  if (!panel || !hint) return;

  if (!debug.open) {
    panel.style.display = "none";
    hint.style.display = "block";
    return;
  }

  hint.style.display = "none";
  panel.style.display = "block";

  let onStyle  = "color:#50dc78; font-weight:bold;";
  let offStyle = "color:#555;";
  let valStyle = "color:#6ab0f5;";
  let dimStyle = "color:#666;";
```

```
 let velRows = debug.showVelocity ? `

   <tr><td style="${dimStyle}">────────────────────</td></tr>

   <tr><td style="${dimStyle}">pos</td><td style="${valStyle}">${nf(player.pos.x,1,1)},
${nf(player.pos.y,1,1)}</td></tr>

   <tr><td style="${dimStyle}">vel</td><td style="${valStyle}">${nf(player.vel.x,1,2)},
${nf(player.vel.y,1,2)}</td></tr>

   <tr><td style="${dimStyle}">gnd</td><td style="${valStyle}">${grounded ? "yes" :
"no"}</td></tr>

   <tr><td style="${dimStyle}">ani</td><td style="${valStyle}">${player.ani.name ??
"-"}</td></tr>

 `: "";



 panel.innerHTML = `

   <table style="border-collapse:collapse; width:100%;">

     <tr>

      <td colspan="2" style="color:#777; padding-bottom:4px; letter-spacing:0.06em;
font-size:10px;">

       DEBUG  <span style="color:#444">[~ to close]</span>

      </td>

     </tr>

     <tr>

      <td style="padding-right:16px;">moon grav <span style="color:#444">[G]</span></td>

      <td style="${debug.moonGravity ? onStyle : offStyle}">${debug.moonGravity ? "ON" :
"off"}</td>

     </tr>

     <tr>
```

```
    <td>hitboxes <span style="color:#444">[H]</span></td>

    <td style="${debug.showHitboxes ? onStyle : offStyle}">${debug.showHitboxes ? "ON" : "off"}</td>

   </tr>

   <tr>

    <td>velocity <span style="color:#444">[V]</span></td>

    <td style="${debug.showVelocity ? onStyle : offStyle}">${debug.showVelocity ? "ON" : "off"}</td>

   </tr>

   ${velRows}

  </table>

 `;

}
```

Finally, in `draw()`, replace the call to `drawDebugOverlay(grounded)` with:

javascript

```
updateDebugPanel(grounded);
```

Why this works: the `"pixelated"` canvas setting applies CSS `image-rendering: pixelated` to the `<canvas>` element itself. By appending the debug panel as a sibling DOM element over the canvas, it renders with the browser's normal subpixel font smoothing and anti-aliasing — completely unaffected by the canvas rendering mode.

const VIEWW = 320, VIEWH = 180; const TILE_W = 24, TILE_H = 24; const FRAME_W = 32, FRAME_H = 32; const MAP_START_Y = VIEWH - TILE_H * 4; const GRAVITY = 10; function preload() { playerImg = loadImage("assets/foxSpriteSheet.png"); bgImg = loadImage("assets/combinedBackground.png"); groundImg = loadImage("assets/groundTile.png"); groundDeepImg = loadImage("assets/groundTileDeep.png"); if (typeof loadSound === "function") {   jumpSfx = loadSound("assets/sfx/jump.wav");   musicSfx = loadSound("assets/sfx/music.wav"); } } function setup() { new Canvas(VIEWW, VIEWH); allSprites.pixelPerfect = true; i removed the pixellated part in createcanvas but now the viewpport is really small, how to make it bigger

10:28 p.m.

The `"pixelated"` argument wasn't controlling the size — it was just a rendering hint. The canvas is small because p5.play's `Canvas` defaults to the literal pixel dimensions you gave it (320×180). You need to scale it up to fill the screen while keeping the pixelated look via CSS.

Replace your canvas line with this:

javascript

```javascript
function setup() {
  let cnv = new Canvas(VIEWW, VIEWH, "pixelated"); // put pixelated back


  // Scale the canvas up to fill the window, preserving aspect ratio
  cnv.elt.style.width = "100%";
  cnv.elt.style.height = "100%";
  cnv.elt.style.objectFit = "contain";
  cnv.elt.style.imageRendering = "pixelated"; // keeps pixel art crisp when scaled


  // rest of setup...
}
```

And add this to your HTML/CSS (or a `<style>` tag in your `index.html`):

css

```css
html, body {
  margin: 0;
  padding: 0;
  width: 100%;
  height: 100%;
```

```
  background: black;

  overflow: hidden;

}


canvas {

  display: block;

  width: 100% !important;

  height: 100% !important;

  object-fit: contain;

}
```

What's happening: the canvas is still internally 320×180 pixels — that's your game's resolution. CSS then stretches it to fill the viewport. `image-rendering: pixelated` tells the browser to use nearest-neighbour scaling when it upscales, which keeps your pixel art crisp instead of blurry. This is the standard approach for pixel art games — low internal resolution, CSS upscale.

The debug panel will also automatically scale with it since it's a DOM element positioned over the canvas.

html, body { margin: 0; padding: 0; width: 100%; height: 100%; background: black; overflow: hidden; } canvas { display: block; width: 100% !important; height: 100% !important; object-fit: contain; } now nothing is showing up

10:30 p.m.

The CSS is fighting p5.play — forcing `width/height: 100%` on the canvas breaks its internal layout. Also `object-fit` doesn't work on `<canvas>`. The right approach is to let p5 size the canvas normally, and use CSS to center it without stretching it.

Replace your CSS with this:

```
css

html, body {
```

```
  margin: 0;
  padding: 0;
  width: 100%;
  height: 100%;
  background: black;
  overflow: hidden;
  display: flex;
  align-items: center;
  justify-content: center;
}

canvas {
  image-rendering: pixelated;
  /* do NOT set width/height here — let JS handle it */
}
```

And in `setup()`, replace the canvas scaling lines with this:

javascript

```
function setup() {
  let cnv = new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;

  // Scale up to fit the window, preserving 320x180 aspect ratio
  let scale = Math.min(
    Math.floor(windowWidth / VIEWW),
```

```
  Math.floor(windowHeight / VIEWH)
 );
 scale = max(scale, 1); // never scale below 1x
 cnv.elt.style.width  = (VIEWW * scale) + "px";
 cnv.elt.style.height = (VIEWH * scale) + "px";

 // ... rest of setup
}

// Also add this so it re-scales if the window is resized
function windowResized() {
 let scale = Math.min(
  Math.floor(windowWidth / VIEWW),
  Math.floor(windowHeight / VIEWH)
 );
 scale = max(scale, 1);
 let cnv = document.querySelector("canvas");
 cnv.style.width  = (VIEWW * scale) + "px";
 cnv.style.height = (VIEWH * scale) + "px";
}
```

`Math.floor` keeps it at whole-number multiples (1×, 2×, 3×, 4×...) so pixels stay perfectly sharp with no half-pixel blurring. On a 1920×1080 screen your 320×180 canvas will render at 5× = 1600×900, crisp and centered on the black background.

You can also remove the three `cnv.elt.style` lines you added previously in `setup()` since this replaces them.

the debug screen is pixellated and illegible make it legible

10:32 p.m.

The only real fix is to move the debug panel out of the canvas entirely and into a DOM overlay, as described earlier. Here's the complete updated file with that fully integrated — just replace your current `sketch.js`:

Viewed a file, created a file, read a file

The key changes:

- **`drawDebugOverlay()` is completely removed** — drawing text on a pixelated canvas will always look bad because the canvas itself has `image-rendering: pixelated` applied, which nearest-neighbour scales every pixel including text.
- **`createDebugDOM()`** runs once in `setup()` and appends two plain `<div>` elements as siblings to the canvas. The browser renders these at full native resolution with antialiased fonts, completely independent of the canvas.
- **`updateDebugDOM(grounded)`** replaces the old overlay call in `draw()` — it updates the panel's `innerHTML` each frame with the latest state.

Also make sure your CSS looks like this (no `width/height` overrides on canvas):

```
css

html, body {

 margin: 0;

 padding: 0;

 width: 100%;

 height: 100%;

 background: black;

 overflow: hidden;

 display: flex;

 align-items: center;

 justify-content: center;
```

}

Sketch

JS

my html html, body { margin: 0; padding: 0; width: 100%; height: 100%; background: black; overflow: hidden; display: flex; align-items: center; justify-content: center; } canvas { display: block; } and the sketch js is what was given but nothing is showing up on the screen

10:35 p.m.

The problem is that p5.play creates the canvas inside a `<main>` element, and that `<main>` has no size, so the flex centering collapses it. Fix the CSS to target `main` as well:

css

```
html, body, main {
  margin: 0;
  padding: 0;
  width: 100%;
  height: 100%;
  background: black;
  overflow: hidden;
  display: flex;
  align-items: center;
  justify-content: center;
}


canvas {
  display: block;
  image-rendering: pixelated;
```

```
}
```

If that still doesn't work, your `index.html` might not be loading the libraries correctly. Here's a known-good full `index.html` to check against:

html

```html
<!DOCTYPE html>

<html>

 <head>

  <meta charset="utf-8" />

  <title>Fox Game</title>

  <script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.4/p5.min.js"></script>

  <script src="https://p5play.org/v3/planck.min.js"></script>

  <script src="https://p5play.org/v3/p5play.js"></script>

  <style>

   html, body, main {

    margin: 0;

    padding: 0;

    width: 100%;

    height: 100%;

    background: black;

    overflow: hidden;

    display: flex;

    align-items: center;

    justify-content: center;

   }
```

```
    canvas {
      display: block;
      image-rendering: pixelated;
    }
  </style>
 </head>
 <body>
  <script src="sketch.js"></script>
 </body>
</html>
```

The two most common culprits when nothing shows at all are a wrong p5play script URL (the library didn't load), or the canvas getting created inside an unsized container (flex collapses it to 0). Opening your browser's dev console (F12) and checking for red errors will immediately tell you which it is.

<!doctype html> <html lang="en"> <head> <meta charset="utf-8" /> <title>GBDA302 Week 9 - Example 3: Adding Sound & Music</title> <style> body { margin: 0; overflow: hidden; background: #222; } canvas { display: block; } </style> </head> <body> <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script> <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script> <script src="https://p5play.org/v3/planck.min.js"></script> <script src="https://p5play.org/v3/p5play.min.js"></script> <script src="sketch.js"></script> </body> </html> the current html

10:35 p.m.

Your HTML is missing `main` in the CSS, and `body` needs to be a flex container. Update just the `<style>` block:

html

```
<style>
 html, body {
```

```
  margin: 0;
  padding: 0;
  width: 100%;
  height: 100%;
  overflow: hidden;
  background: #222;
 }

 /* p5play wraps the canvas in a <main> tag */
 main {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  height: 100%;
  background: black;
 }

 canvas {
  display: block;
  image-rendering: pixelated;
 }
</style>
```

That's the only change needed — everything else in your HTML is correct. The canvas was rendering but `<main>` had no size so it collapsed to nothing.

<!doctype html> <html lang="en"> <head> <meta charset="utf-8" /> <title>GBDA302 Week 9 - Example 3: Adding Sound & Music</title> <style> html, body { margin: 0; padding: 0; width: 100%; height: 100%; overflow: hidden; background: #222; } /* p5play wraps the canvas in a <main> tag */ main { display: flex; align-items: center; justify-content: center; width: 100%; height: 100%; background: black; } canvas { display: block; image-rendering: pixelated; } </style> </head> <body> <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script> <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script> <script src="https://p5play.org/v3/planck.min.js"></script> <script src="https://p5play.org/v3/p5play.min.js"></script> <script src="sketch.js"></script> </body> </html> the screen is still not loading

10:36 p.m.

Open your browser's dev console (`F12` → Console tab) and look for red errors — that will pinpoint exactly what's failing. But the most likely issue is that `createDebugDOM()` is crashing because it runs in `setup()` before `<main>` and the canvas exist in the DOM.

Try this quick test — strip `sketch.js` down to just this to confirm p5 itself is working:

javascript

```
function setup() {

 new Canvas(320, 180);

}



function draw() {

 background(255, 0, 0);

}
```

If a red rectangle appears, p5 is fine and the bug is in the full sketch. If nothing appears, the libraries aren't loading.

If p5 is fine, the fix is to defer `createDebugDOM()` until after the canvas is in the DOM. In `setup()`, wrap it in a `setTimeout`:

javascript

```javascript
function setup() {
  let cnv = new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;
  world.gravity.y = NORMAL_GRAVITY;

  let scale = Math.max(1, Math.min(
    Math.floor(windowWidth / VIEWW),
    Math.floor(windowHeight / VIEWH)
  ));
  cnv.elt.style.width  = (VIEWW * scale) + "px";
  cnv.elt.style.height = (VIEWH * scale) + "px";

  // Defer DOM panel creation until after p5play finishes building the page
  setTimeout(createDebugDOM, 0);

  // ... rest of setup unchanged
}
```

Also add a guard at the top of `updateDebugDOM` so it silently skips if the panel isn't ready yet:

javascript

```javascript
function updateDebugDOM(grounded) {
  let panel = document.getElementById("dbg-panel");
  let hint  = document.getElementById("dbg-hint");
  if (!panel || !hint) return; // not ready yet, skip
```

```
  // ... rest unchanged
}
```

The `setTimeout(..., 0)` pushes `createDebugDOM` to the next event loop tick, after p5play has finished appending `<main>` and the canvas to the document, so `querySelector("canvas").parentElement` correctly finds `<main>` instead of returning `null` and throwing.

Uncaught TypeError: Cannot read properties of undefined (reading 'style') at setup (sketch.js:90:11) at _setup (p5.min.js:2:466572) at _runIfPreloadsAreDone (p5.min.js:2:465625) at g._decrementPreload (p5.min.js:2:465830) at p5.sound.min.js:2:198922 at AudioContext.<anonymous> (p5.sound.min.js:2:102726) the error

10:37 p.m.

The error is on the canvas scaling lines — `new Canvas()` in p5play doesn't return the canvas object the same way as regular p5. Fix it by grabbing the canvas element directly instead:

javascript

```javascript
function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;
  world.gravity.y = NORMAL_GRAVITY;


  // Grab the canvas element directly instead of from the return value
  let cnvEl = document.querySelector("canvas");
  if (cnvEl) {
    let scale = Math.max(1, Math.min(
      Math.floor(windowWidth / VIEWW),
      Math.floor(windowHeight / VIEWH)
    ));
```

```
  cnvEl.style.width  = (VIEWW * scale) + "px";
  cnvEl.style.height = (VIEWH * scale) + "px";
 }

 setTimeout(createDebugDOM, 0);

 // ... rest of setup unchanged
}
```

And update `windowResized` the same way since it already does this correctly:

javascript

```
function windowResized() {
 let scale = Math.max(1, Math.min(
  Math.floor(windowWidth / VIEWW),
  Math.floor(windowHeight / VIEWH)
 ));
 let cnvEl = document.querySelector("canvas");
 if (cnvEl) {
  cnvEl.style.width  = (VIEWW * scale) + "px";
  cnvEl.style.height = (VIEWH * scale) + "px";
 }
}
```

The `if (cnvEl)` guard prevents the crash in case the canvas isn't in the DOM yet at the moment `setup()` runs.

sketch.js:106 Uncaught ReferenceError: cnv is not defined at setup (sketch.js:106:3) at _setup (p5.min.js:2:466572) at _runIfPreloadsAreDone (p5.min.js:2:465625) at g._decrementPreload (p5.min.js:2:465830) at p5.sound.min.js:2:198922 at AudioContext.<anonymous> (p5.sound.min.js:2:102726)

10:38 p.m.

You still have the old `cnv.elt.style...` lines in your setup from a previous version. Can you share your current full `sketch.js`? In the meantime, search for any line that says `cnv.` and delete it — those are the leftover references. Your setup should have no mention of `cnv` at all now, just:

javascript

```javascript
function setup() {
 new Canvas(VIEWW, VIEWH, "pixelated");  // no "let cnv ="
 allSprites.pixelPerfect = true;
 world.gravity.y = NORMAL_GRAVITY;


 let cnvEl = document.querySelector("canvas");
 if (cnvEl) {
  let scale = Math.max(1, Math.min(
   Math.floor(windowWidth / VIEWW),
   Math.floor(windowHeight / VIEWH)
  ));
  cnvEl.style.width  = (VIEWW * scale) + "px";
  cnvEl.style.height = (VIEWH * scale) + "px";
 }


 setTimeout(createDebugDOM, 0);
```

```
 if (musicSfx) musicSfx.setLoop(true);

 startMusicIfNeeded();


 // ... rest of setup

}
```

10:39 p.m.

I can see the problem — you have `cnv.elt.style` lines still in `setup()` around line 90, right after the `setTimeout`. Delete those three lines:

javascript

```
// DELETE THESE THREE LINES (they're duplicates and cnv is not defined):

 cnv.elt.style.width = VIEWW * scale + "px";

 cnv.elt.style.height = VIEWH * scale + "px";
```

Also, `createDebugDOM()` is being called twice — once via `setTimeout` and once directly at the bottom of `setup()`. Remove the direct call at the bottom:

javascript

```
// DELETE THIS LINE near the bottom of setup():

 createDebugDOM();
```

Your `setup()` should look exactly like this:

javascript

```
function setup() {

 new Canvas(VIEWW, VIEWH, "pixelated");

 allSprites.pixelPerfect = true;

 world.gravity.y = NORMAL_GRAVITY;
```

```
let cnvEl = document.querySelector("canvas");
if (cnvEl) {
  let scale = Math.max(1, Math.min(
    Math.floor(windowWidth / VIEWW),
    Math.floor(windowHeight / VIEWH)
  ));
  cnvEl.style.width  = VIEWW * scale + "px";
  cnvEl.style.height = VIEWH * scale + "px";
}

setTimeout(createDebugDOM, 0);

if (musicSfx) musicSfx.setLoop(true);
startMusicIfNeeded();

ground = new Group();
ground.physics = "static";
ground.img = groundImg;
ground.tile = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundDeepImg;
```

```
groundDeep.tile = "d";

new Tiles(level, 0, 0, TILE_W, TILE_H);

player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -4;
player.addAnis(playerAnis);
player.ani = "idle";
player.w = 18;
player.h = 20;
player.friction = 0;
player.bounciness = 0;

sensor = new Sprite();
sensor.x = player.x;
sensor.y = player.y + player.h / 2;
sensor.w = player.w;
sensor.h = 2;
sensor.mass = 0.01;
sensor.removeColliders();
```

```
 sensor.visible = false;

 let sensorJoint = new GlueJoint(player, sensor);

 sensorJoint.visible = false;

}
```

how ot change the colour of the [G] in the debug panel

10:43 p.m.

Find this section in `updateDebugDOM` and change the hex colour in the `[G]` span:

javascript

```
   <tr>

     <td ${lbl}>moon grav <span style="color:#ff9944">[G]</span></td>

     <td ${debug.moonGravity ? on : off}>${debug.moonGravity ? "ON" : "off"}</td>

    </tr>
```

Just replace `#ff9944` with whatever colour you want. The other key hints `[H]` and `[V]` are in the same format right below — they all currently use `color:#3a3a3a` (dark gray), so change whichever ones you want the same way.